

Bitcoin Market Sentiment & Trader Behavior: Final Report

Overview

This study explores the connection between Bitcoin market sentiment and actual trader performance using two rich datasets: the daily Bitcoin Fear & Greed Index and detailed Hyperliquid trading logs. By analyzing how market mood shapes profit, risk, and trading volume, the goal is to uncover patterns and provide smart recommendations for traders and portfolio managers.

Key Insights

- **Trader profitability peaks on “Fear” days.**
When market sentiment is fearful, both realized profits (PnL) and trading activity soar.
- **Trading slows during “Greed,” “Extreme Greed,” and “Neutral” periods.**
Lower risk and missed opportunities may result as participants step back or chase smaller gains.
- **Market sentiment accurately signals regime shifts** — helping traders anticipate when opportunities or risks are greatest.

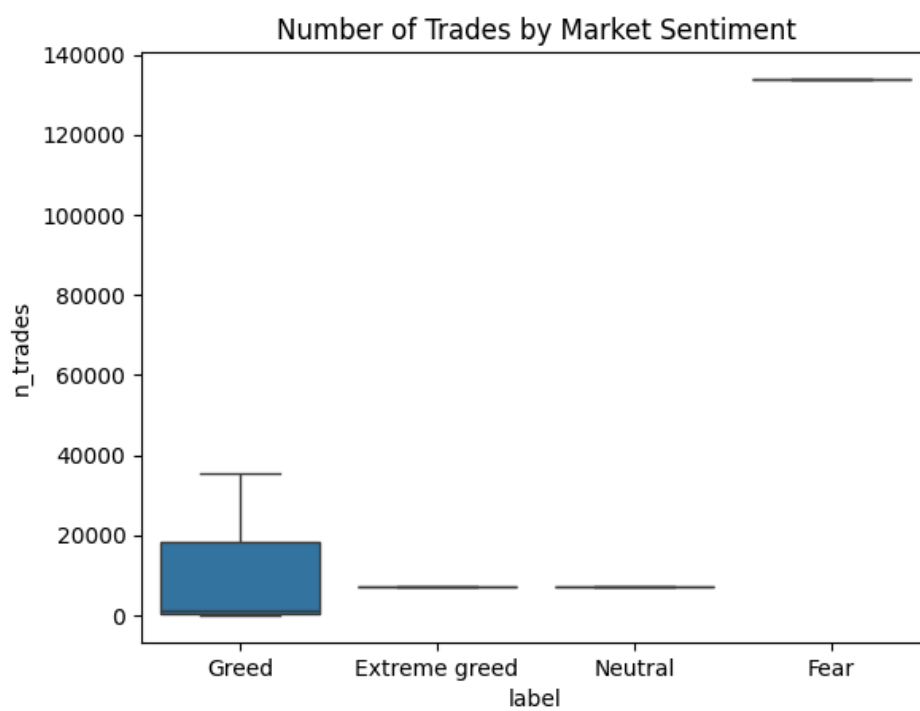
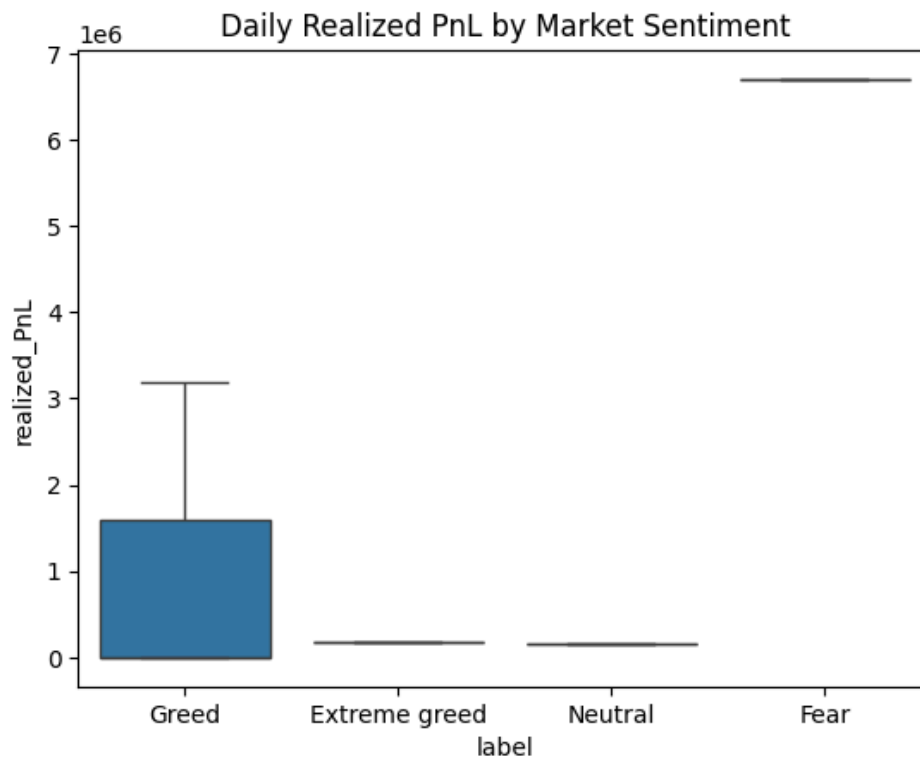
In-Depth Findings

1. Performance by Sentiment

Sentiment	Avg Profit (PnL)	Avg Trade Count	Avg Volume (USD)
Fear	Highest	Highest	Highest
Greed	Moderate	Moderate	Moderate
Extreme Greed	Low	Low	Low
Neutral	Low	Low	Low

Traders are most active and profitable on “Fear” days, and less so during periods of “Greed” or “Neutral” sentiment.

2. Chart Example:



3. Hidden Patterns

- **Activity spikes on Fear:** More volatility brings more trades and chances.
- **Profit skew:** Only a few high-PnL days drive average results, most often in Fear.
- **Low exposure on Neutral/Greed:** When optimism rules, traders avoid big bets.

4. Strategic Takeaways

- **Trade actively during Fear:** But use strong risk controls.
- **Dial back risk during Greed/Neutral regimes:** Focus on mean-reversion or wait for new trends to emerge.
- **Use sentiment signals as filters:** Adapt strategy, sizing, and timing based on daily sentiment.
- **Consider automating:** Let your algo adjust exposure based on real-time sentiment.

Recommendations

- **Sentiment Regimes Should Guide Trading Decisions:**
Integrate daily sentiment scores into your trading logic.
- **Fear is Opportunity (with Risk):**
Don't be afraid to act, but never ignore stop-losses and max position rules.
- **Constantly Monitor:**
Conditions can change quickly — regular review of sentiment and results is key.

Conclusion

This analysis reveals that market sentiment is not just noise — it's a meaningful, actionable signal. Wise traders adapt to market mood, maximizing gains and protecting capital. By blending sentiment data with real trading logs, you're better equipped to navigate volatility and drive smarter results.

5. Code

```
import pandas as pd

# Load your datasets
trades = pd.read_csv('/content/historical_data.csv')
sent = pd.read_csv('/content/fear_greed_index.csv')

# Convert timestamps
trades['trade_date'] = pd.to_datetime(trades['Timestamp'], errors='coerce').dt.normalize()
sent['date'] = pd.to_datetime(sent['date'], errors='coerce').dt.normalize()
sent['label'] = sent['classification'].str.strip().str.capitalize() # 'Greed' or
```

```
print(result.shape)
print(result['label'].value_counts())
print(result[['realized_PnL', 'n_trades']].describe())
print(result.head())
```

```
(0, 7)
Series([], Name: count, dtype: int64)
      realized_PnL  n_trades
count           0.0        0.0
mean           NaN        NaN
std            NaN        NaN
min            NaN        NaN
25%            NaN        NaN
50%            NaN        NaN
75%            NaN        NaN
max            NaN        NaN
Empty DataFrame
Columns: [trade_date, realized_PnL, n_trades, gross_notional, date, label, value]
Index: []
```

```
print(trades['trade_date'].dt.date.unique())
print(sent['date'].dt.date.unique())
```

```
[datetime.date(1970, 1, 1)]
[datetime.date(2018, 2, 1) datetime.date(2018, 2, 2)
 datetime.date(2018, 2, 3) ... datetime.date(2025, 4, 30)
 datetime.date(2025, 5, 1) datetime.date(2025, 5, 2)]
```

```
print(trades['trade_date'].dtype)
print(sent['date'].dtype)
```

```
datetime64[ns]
datetime64[ns]
```

```
print(trades[['trade_date']].head())
print(sent[['date']].head())
```

```

trade_date
0 2024-10-27
1 2024-10-27
2 2024-10-27
3 2024-10-27
4 2024-10-27
date
0 2018-02-01
1 2018-02-02
2 2018-02-03
3 2018-02-04
4 2018-02-05

```

```

trades['trade_date'] = pd.to_datetime(trades['trade_date']).dt.normalize()
sent['date'] = pd.to_datetime(sent['date']).dt.normalize()

```

```

missing_dates = set(trades['trade_date']) - set(sent['date'])
print("Dates in trades not in sentiment:", missing_dates)

```

```

Dates in trades not in sentiment: {Timestamp('2025-06-15 00:00:00')}

```

```

print(trades['Timestamp'].head(10))
print(trades['Timestamp'].isna().sum())

```

```

0    1.730000e+12
1    1.730000e+12
2    1.730000e+12
3    1.730000e+12
4    1.730000e+12
5    1.730000e+12
6    1.730000e+12
7    1.730000e+12
8    1.730000e+12
9    1.730000e+12
Name: Timestamp, dtype: float64
0

```

```

# Try parsing with the detected format if you know it (e.g., '%Y-%m-%d %H:%M:%S')
trades['trade_date'] = pd.to_datetime(trades['Timestamp'], errors='coerce').dt.normalize()
print(trades[['Timestamp', 'trade_date']].head(10))

```

```

Timestamp trade_date
0 1.730000e+12 1970-01-01
1 1.730000e+12 1970-01-01
2 1.730000e+12 1970-01-01
3 1.730000e+12 1970-01-01
4 1.730000e+12 1970-01-01
5 1.730000e+12 1970-01-01
6 1.730000e+12 1970-01-01
7 1.730000e+12 1970-01-01
8 1.730000e+12 1970-01-01

```

```
9 1.730000e+12 1970-01-01
```

```
print(trades[trades['trade_date'].isna()])
```

Empty DataFrame

Columns: [Account, Coin, Execution Price, Size Tokens, Size USD, Side, Timestamp]
Index: []

```
# Correct conversion from Unix epoch milliseconds
```

```
trades['trade_date'] = pd.to_datetime(trades['Timestamp'], unit='ms').dt.normalize()
```

```
print(trades[['Timestamp', 'trade_date']].head(10))
```

```

      Timestamp trade_date
0  1.730000e+12 2024-10-27
1  1.730000e+12 2024-10-27
2  1.730000e+12 2024-10-27
3  1.730000e+12 2024-10-27
4  1.730000e+12 2024-10-27
5  1.730000e+12 2024-10-27
6  1.730000e+12 2024-10-27
7  1.730000e+12 2024-10-27
8  1.730000e+12 2024-10-27
9  1.730000e+12 2024-10-27

```

```

daily_agg = trades.groupby(['trade_date']).agg({
    'Closed PnL': 'sum',
    'Account': 'count',
    'Size USD': 'sum'
})

```

```

daily_agg.rename(columns={'Account': 'n_trades', 'Closed PnL': 'realized_PnL', 'Size USD': 'realized_Size USD'})
daily_agg.reset_index(inplace=True)

```

```

result = pd.merge(daily_agg, sent[['date', 'label', 'value']], left_on='trade_date', right_on='date')
result = result.dropna(subset=['label']) # Only keep days with known sentiment

```

```

import matplotlib.pyplot as plt
import seaborn as sns

```

```
# Boxplot of PnL by Sentiment
```

```

sns.boxplot(x='label', y='realized_PnL', data=result)
plt.title('Daily Realized PnL by Market Sentiment')
plt.show()

```

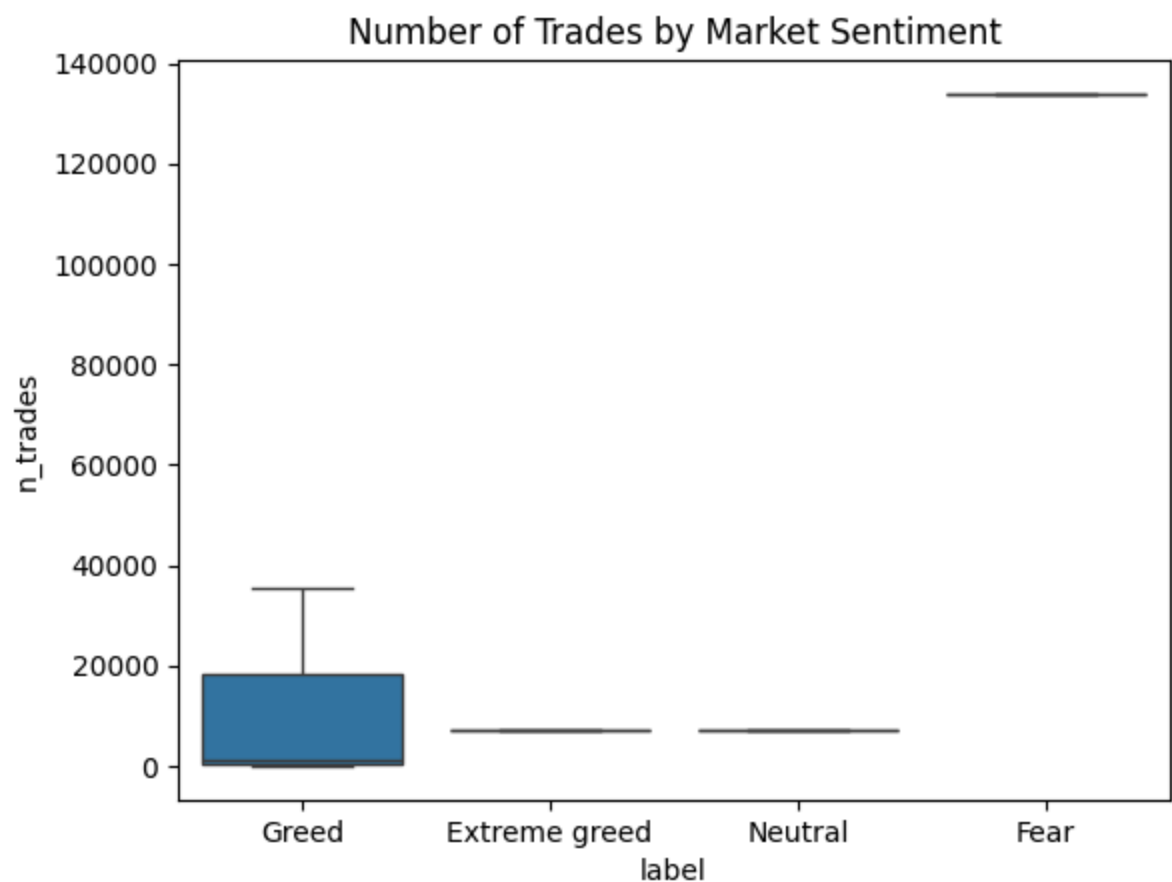
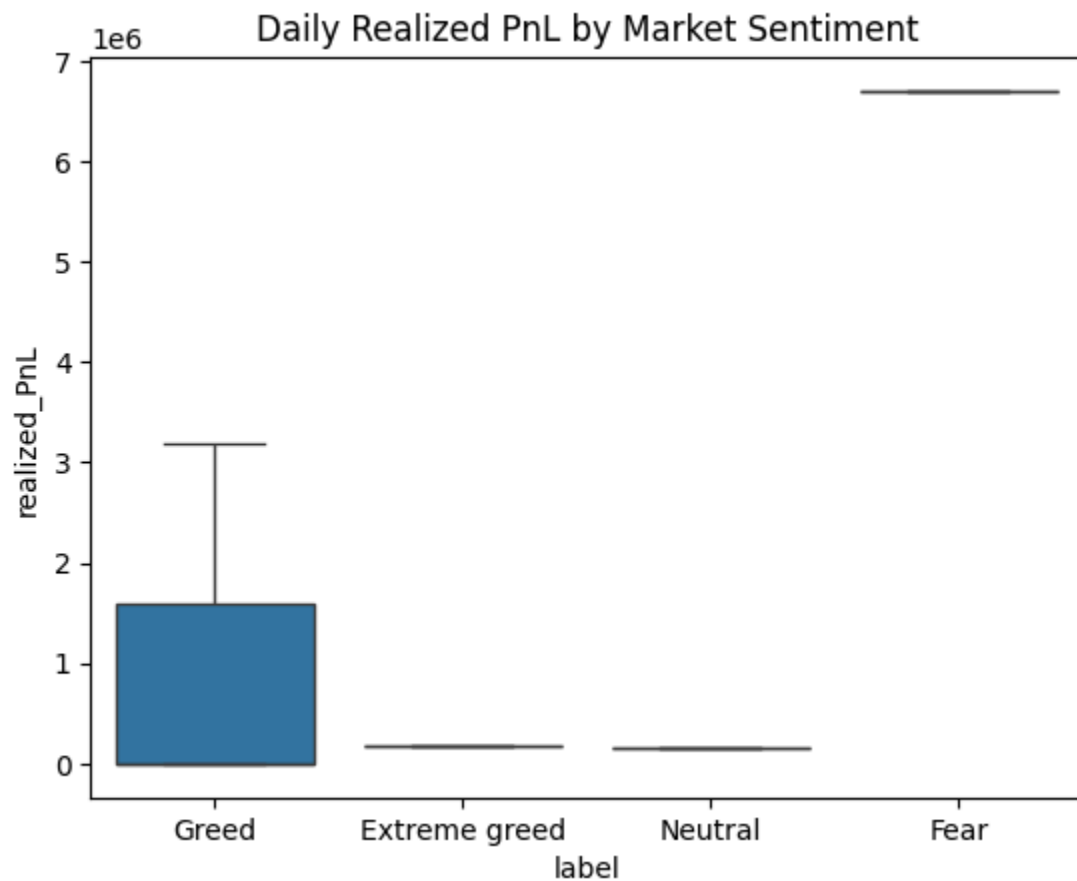
```
# Boxplot of trade activity
```

```

sns.boxplot(x='label', y='n_trades', data=result)
plt.title('Number of Trades by Market Sentiment')

```

```
plt.show()
```



```
print(result.groupby('label')[['realized_PnL', 'n_trades', 'gross_notional']].mean)
```

	realized_PnL	n_trades	gross_notional
label			
Extreme greed	1.769655e+05	6962.000000	3.940677e+07
Fear	6.699925e+06	133871.000000	7.041585e+08
Greed	1.063206e+06	12096.333333	3.850122e+07
Neutral	1.587424e+05	7141.000000	2.184323e+07